CSEN-140 PR1: Text Classification — Full Technical Write-Up

How I built an 11-model k-NN ensemble for news classification — combining surface n-grams, latent semantic analysis (LSI), and pseudo-relevance feedback (PRF) — maximized GPU utilization on an RTX 5080, and iteratively refined score-level ensembling through trial and error. Best leaderboard score: 0.9574 (8-model). Current local F1: 0.9245 (11-model, score-level). Leaderboard result pending.

PART 1 — THE ASSIGNMENT AND THE GOAL

The assignment (PR1) requires building a k-Nearest Neighbours (k-NN) text classifier from scratch — no sklearn KNeighborsClassifier allowed. The dataset is a collection of news abstracts belonging to 4 classes (e.g. World, Sports, Business, Science/Tech). The goal is to maximize macro F1-score on a held-out test set, measured on a class leaderboard with 5 submissions per 24 hours.

The core idea of k-NN is: given a new document, find the k most similar training documents, and let them vote on the label. The tricky parts are (1) how you represent documents as vectors, and (2) how you measure similarity efficiently when you have 100,000+ training examples and a tight submission budget.

PART 2 — HOW DOCUMENTS BECOME VECTORS

2.1 Tokenization and Preprocessing

Raw text can't go directly into math. We first convert each abstract into a list of tokens. For most models we:

- Lowercase everything.
- Strip punctuation (using str.maketrans).
- Remove stopwords ("the", "is", "at", etc.) — words so common they carry no signal.
- Apply a stemmer or lemmatizer to collapse word variants.

Three different normalization strategies were used across models:

- Porter Stemmer — heuristically chops suffixes: "running" → "run", "prices" → "price".
- WordNet Lemmatizer — looks up the dictionary base form: "better" → "good".
- Raw words — no normalization at all; keeps "running", "runner", "runs" separate.

We also generate n-grams — sequences of consecutive tokens that capture phrases. "stock market" as a bigram is more informative than "stock" and "market" separately.

- Unigrams: single words ("oil", "price").
- Bigrams: two-word phrases ("oil_price", "stock_market").
- Trigrams: three-word phrases ("stock_market_crash") — used in Models 1 and 8.

2.2 Bag of Words — Turning Tokens into Numbers

Once we have tokens, we build a vocabulary: a mapping from every unique token to an integer index. Then each document becomes a sparse vector: index i holds the weight of word i in that document. Most entries are zero (a document only contains a tiny fraction of all vocabulary words), so we use scipy sparse CSR matrices to store only the non-zero entries — this is critical for memory efficiency.

2.3 BM25 Weighting

Not all word occurrences are equally informative. BM25 (Best Match 25) is the gold standard weighting scheme from information retrieval. The formula for term t in document d is:

```
score(t, d) = IDF(t) × tf × (k1 + 1) / (tf + k1 × (1 - b + b × |d| /
avgdl))
```

Where:

- IDF(t) = log((N - df + 0.5) / (df + 0.5) + 1) — inverse document frequency. Rare words get high IDF; extremely common words get low IDF.
- tf = term frequency in document d.
- |d| = document length, avgdl = average document length across all docs.
- k1 controls term frequency saturation (we found k1=1.2 optimal).
- b controls length normalization (we found b=0.5 optimal).

After computing BM25 scores, every row (document) is L2-normalized (divided by its Euclidean length). This means cosine similarity between two document vectors is simply their dot product — which we can compute very fast.

2.4 TF-IDF Weighting (Alternative)

For some models we use sublinear TF-IDF instead of BM25:

```
score(t, d) = (1 + log(tf)) / total_terms × IDF(t)
```

This compresses large term frequencies logarithmically. BM25 tends to outperform TF-IDF for k-NN because its length normalization is more principled, but TF-IDF adds useful diversity in an ensemble. The key difference: BM25's saturation term dampens high-frequency words more aggressively than sublinear TF-IDF, so the two scoring functions sometimes disagree on borderline documents — which is exactly what makes combining them valuable.

2.5 Chi-Squared Feature Selection

Even after capping vocabulary at 150,000–300,000 words, many features are noise. Chi-squared (χ^2) feature selection measures the statistical dependence between each feature and the class labels. We keep only the top k features by χ^2 score — 200,000 for Models 1 and 8, 120,000 for most others, 50,000 for Model 3. This removes noise, reduces memory, and often improves accuracy because k-NN is sensitive to irrelevant dimensions.

PART 3 — K-NN CLASSIFICATION

3.1 The Algorithm

After building normalized sparse matrices for training and test, k-NN works as follows for each test document:

- Compute cosine similarity to every training document (dot product, since both are L2-normalized).
- Find the top-k most similar training documents.
- Let those k neighbors vote on the label, weighted by their similarity score.
- Predict the label with the highest total weight.

Distance-weighted voting means a neighbor with similarity 0.95 has much more influence than one with similarity 0.60 — this consistently outperforms simple majority voting.

3.2 Choosing k — The Journey

k is a critical hyperparameter. Too small: predictions are noisy. Too large: you start including documents from the wrong class. We sweep over a list of k values and pick the one that maximizes validation F1. Our K_LIST evolved through several iterations:

- Initial: K_LIST = [3, 5, 7, 9, 11] — only small k values, first baseline.
- Expanded: K_LIST = list(range(21, 42)) — found larger k values were better, but accidentally dropped k=11.
- Bug fix: K_LIST = [11] + list(range(21, 42)) — re-added k=11 explicitly after score dropped.
- Attempted extension: K_LIST = [3, 5, 7, 9, 11] + list(range(21, 42)) — added small k values back to test all options.
- Reverted: K_LIST = [11] + list(range(21, 42)) — the extended sweep caused overfitting on the leaderboard. Final version.

We skip k=13–19 because experiments showed k=11 and the k=21–41 range tend to be optimal — the middle range often underperforms both. The range 21–41 is swept to find the exact best odd value (even k values can cause ties, so odd is preferred).

PART 4 — THE ENSEMBLE: 11 MODELS

A single k-NN model is limited by its feature representation. An ensemble combines multiple models, each with a different "view" of the data. When one model makes a mistake, the others may still get it right — and the weighted vote corrects the error.

Each model is weighted by its validation F1 score, so stronger models have more influence in the final vote.

4.1 The 11 Final Models

M1: BM25 + unigrams + bigrams + trigrams + chi2(200k), vocab 300k

→ Porter stemmer. The kitchen-sink model — largest feature set, includes trigrams. Started at vocab 200k / chi2(150k), expanded to 300k / 200k to capture more rare trigrams like "interest_rate_hike". Individual F1: ~0.9216.

M2: TF-IDF + unigrams + bigrams + chi2(120k)

→ Porter stemmer. Same preprocessing as M1 but TF-IDF scoring, no trigrams. Individual F1: ~0.9169.

M3: BM25 + unigrams only + chi2(50k)

→ Porter stemmer. Unigrams only — completely ignores phrase structure. Weakest individual model (~0.9135) but consistently helps the ensemble on the test set. We tried removing it — local ensemble improved slightly (0.9201→0.9204) but the leaderboard dropped (0.9574→0.9573). It captures something the other models miss.

M4: BM25 + unigrams + bigrams + chi2(120k)

→ Porter stemmer. Like M1 but no trigrams, smaller feature set. Individual F1: ~0.9180.

M5: BM25 + unigrams + bigrams + chi2(120k)

→ WordNet lemmatizer. Same scoring as M4 but dictionary-form normalization instead of heuristic stemming. Individual F1: ~0.9178.

M6: BM25 + unigrams + bigrams + chi2(120k)

→ Raw words — NO stemming or lemmatizing. Keeps "stocks", "markets", "running" as-is. Completely different vocabulary from M1–M5. Individual F1: ~0.9181.

M7: TF-IDF + unigrams + bigrams + chi2(120k)

→ WordNet lemmatizer + TF-IDF scoring. Combines M5's preprocessing with M2's scoring. Individual F1: ~0.9170.

M8: TF-IDF + unigrams + bigrams + trigrams + chi2(200k), vocab 300k

→ Porter stemmer + TF-IDF scoring. The trigram counterpart to M1 — same features, different scoring function. BM25 and TF-IDF handle term frequency saturation differently, so they disagree on borderline documents and complement each other. Individual F1: ~0.9209. Adding M8 pushed ensemble from 0.9203 → 0.9211.

M9: LSI (TruncatedSVD, 300 dims) on TF-IDF + trigrams, dense cosine k-NN

→ First latent-semantic model. Takes the pre-chi2 TF-IDF trigram matrix and reduces it to 300 dense dimensions via randomized truncated SVD. Captures latent topic structure — synonyms and topically-related terms end up close in the reduced space even when they never co-occur in the same document. Operates in a fundamentally different representation than M1–M8 (dense 300-dim vs. sparse 100k+). Explained variance: ~0.10. Individual F1: ~0.8813 (lower solo — 300 dims cannot match 200k rich n-gram features), but adds ensemble diversity.

M10: LSI (TruncatedSVD, 300 dims) on BM25 + trigrams

→ Second latent-semantic model. Same SVD-300 reduction but applied to BM25-scored features instead of TF-IDF. BM25's saturation function emphasizes different term-document structure, so the latent topics SVD extracts are slightly different. Individual F1: ~0.8860.

Adds marginal diversity over M9 — the two LSI models are somewhat correlated since they operate on similar underlying n-gram space.

M11: Pseudo-Relevance Feedback (PRF) on M1 — query expansion + 2nd-pass k-NN

→ Rocchio-style query expansion. For each query, the first k-NN pass finds the top-N nearest neighbors (similarity-weighted), the query is replaced by alpha*query + beta*centroid(top-N neighbors), and a second k-NN pass retrieves against the expanded query. Conservative tuning (alpha=0.9, beta=0.1, N=10) was required — aggressive values (alpha=0.7, beta=0.3, N=20) caused query drift to wrong-class clusters. Individual F1: ~0.9202 (just below M1's 0.9216 — PRF does not improve M1 solo, but the 2nd-pass k-NN produces different enough predictions to help the ensemble).

The key insight: every model uses a different combination of (preprocessing × scoring × n-gram order × reduction × query transformation). When they disagree, the ensemble votes. When they all agree, confidence is high. The diversity is what makes ensembles work.

4.2 Latent Semantic Indexing (LSI) — M9 and M10

M1–M8 all operate in sparse n-gram space. Two documents that discuss the same topic using different words (e.g. "stocks rise" vs "shares climb") may share very few n-grams and look dissimilar despite meaning the same thing. LSI addresses this: Truncated Singular Value Decomposition (SVD) factors the term-document matrix into a much smaller dense representation where each dimension captures a latent topic — a weighted combination of terms that tend to co-occur.

Concretely, for M9 we take the TF-IDF + trigram matrix (102080 × 300000) and compute a rank-300 randomized SVD, producing a 102080 × 300 dense matrix. Each row is a 300-dim topic signature of the document. After L2-normalization, cosine similarity (dot product) in this dense space captures semantic closeness rather than surface word overlap. k-NN in the reduced space runs via dense GPU matmul — no sparse math needed since 300 dims × 4 bytes × 102k docs is only 120 MB on the GPU.

Why 300 dimensions? A sweet spot: small enough that topics are meaningful and retrieval is fast, large enough to preserve class structure. Larger components (500, 1000) add marginal signal but also noise. The explained-variance ratio for M9 was ~0.10 — 10% of the total variance captured in 300 dims — low by classic LSI standards because news text has very long-tailed vocabulary, but the retained variance is the class-discriminative part, which is what matters.

M9 alone scores 0.8813, clearly below M1–M8's sparse models. That is expected: 300 latent dims cannot rival 200k explicit features for nearest-neighbor retrieval on this dataset. The value of LSI is purely ensemble diversity — it makes different mistakes than the sparse models because its similarity metric is fundamentally different.

M10 is the same approach applied to the BM25 matrix instead of TF-IDF. It scores 0.8860 solo — slightly better than M9 because BM25's saturation produces a cleaner topic structure. But the gain over M9 in the ensemble was marginal (+0.0001), because both LSI

views cover similar latent space. One LSI model is probably sufficient; two was a small bonus.

4.3 Pseudo-Relevance Feedback (PRF) — M11

PRF is an information-retrieval technique that assumes the top-N retrieved documents for a query are "relevant" (even without ground truth labels), then uses them to refine the query. The classic Rocchio formula:

```
expanded_query = alpha * original_query + beta * centroid(top_N_neighbors)
```

The refined query is then re-issued against the corpus, producing a second set of nearest neighbors that (in theory) are more topically coherent with the query's actual class. The 2-pass approach effectively "pulls" noisy queries toward their topical cluster before final ranking.

Parameter tuning was critical. Initial values (alpha=0.7, beta=0.3, N=20) caused drift into neighbor classes — M11 scored only 0.9182 solo, and the ensemble F1 dropped from 0.9240 to 0.9236. The expanded queries were pulled too strongly toward the centroid, which occasionally straddled class boundaries.

We tuned to conservative drift (alpha=0.9, beta=0.1, N=10): the original query dominates, and only the 10 closest neighbors contribute a small nudge. M11 rose to 0.9202 solo (still slightly below M1's 0.9216 — PRF does not improve M1 directly), but the ensemble climbed from 0.9240 → 0.9245. The 2-pass k-NN produces predictions that are just different enough from M1 to contribute diversity, even though M11 is worse on its own. Classic ensemble behavior.

Why PRF barely helped: news article texts are relatively clean and topically coherent already. Query expansion shines when queries are short, ambiguous, or out-of-distribution — conditions this corpus does not match. Still, the small gain is real and directly attributable to PRF's distinct retrieval mechanism.

4.4 Models We Tried and Removed

Not every model made the final ensemble. One was added and removed after leaderboard testing:

Original M8 (char n-grams): BM25 + character 3–4-grams + chi2(80k)

→ Extracted character sequences from stemmed words using boundary markers: "science" → "#sc", "sci", "cie", "ien", "enc", "nce", "ce#", "scie", "cien", "ienc", "ence", "nce#". Captures morphological patterns across word forms — "scien" appears in "science", "scientific", "scientist" — that no word-level model can see. This was the only model operating below the word level.

→ Result: Local ensemble improved from 0.9201 → 0.9212 (best local score yet). But leaderboard dropped from 0.9570 → 0.9567. Classic overfitting to the validation split — the char n-gram features were highly tuned to the 20% validation set but did not generalize to the held-out test set. Removed permanently.

4.5 Ensemble Weighting Experiments

We also experimented with how to weight the models in the ensemble vote:

- Linear F1 weights (current): w = best_f1_mX. Simple and robust. Gives models proportional influence based on their validation performance.
- Squared F1 weights (tried, reverted): w = best_f1_mX ** 2. Intended to amplify stronger models more aggressively. Made zero difference in practice — the individual F1 scores are too tightly clustered (0.913–0.922) for squaring to actually flip any votes. Reverted.

4.6 Score-Level Ensembling (Soft Probabilities)

After adding M9 and M10, we realized the hard-vote ensemble was throwing away information. The original scheme was: each model produced a single predicted class, and those class votes were summed weighted by F1. That collapses each model's confidence — a model that is 95% sure of class 2 and a model that is 51% sure of class 3 both cast a single vote of equal magnitude.

The fix: have each model return per-class similarity sums (shape n_test × 4) before argmax. L1-normalize each row to turn the sums into a probability distribution, weight-sum across models by their F1 scores, then argmax the combined probabilities. Confident models now contribute more to the decision than unsure ones, on a per-query basis.

```
probs_m = l1_normalize(per_class_similarity_sums_m)  # shape (n_test, 4) —
one per model
combined = sum(w_m * probs_m for m in models)        # weighted soft vote
predictions = argmax(combined, axis=1)
```

Jump: 0.9220 → 0.9240 (+0.0020). Bigger gain than adding any individual model at that point. Implementing this required new helpers — knn_predict_scores and knn_predict_dense_scores — which return the per-class sum matrix instead of hard labels. Vectorized via a mask-and-sum idiom for speed: scores[:, c] = (top_sims * (top_labels == c)).sum(axis=1) — one matmul-free operation per class.

4.7 Temperature Sharpening Experiment

A natural extension of score-level ensembling: raise each model's probability distribution to a power T before combining. T=1 preserves the distribution; T > 1 sharpens it (confident models dominate, unsure models contribute less); T < 1 flattens it. This is equivalent to a tempered softmax.

We swept T ∈ {1.0, 1.5, 2.0, 3.0, 5.0, 7.0, 10.0} on the validation set:

- At 10 models (before M11): T=1.0 and T=2.0 both hit 0.9240 exactly. Higher T hurt. No meaningful gain.
- At 11 models (after M11): T=1.5 became best at 0.9245, marginally above T=1.0.

Interpretation: the F1 weights are already doing the right calibration. Sharpening trades diversity for confidence, which only helps when the weakest models are adding noise the

weights are not filtering out. At 11 models with F1 weights in [0.88, 0.92], the system is near-optimal as-is.

PART 5 — GPU ACCELERATION: THE FULL STORY

5.1 Why GPU?

The core operation in k-NN is: for each test document, compute its similarity to ALL 100,000+ training documents. With 25,520 test documents and 102,080 training documents, that is 2.6 billion similarity computations per model, per k value. We sweep 22 values of k per model, across 8 models — the total is enormous. On CPU this would take many hours. On GPU it takes minutes.

Because our vectors are L2-normalized, cosine similarity = dot product. The full similarity matrix for a batch is: train_matrix @ batch.T. This is a matrix multiplication — exactly what GPUs are designed to do at maximum speed.

5.2 How a GPU Works (Simplified)

A CPU has ~8–16 cores, each very fast and optimized for complex sequential logic. A GPU has thousands of smaller cores (the RTX 5080 Laptop has 10,752 CUDA cores) designed to do the same simple operation on thousands of numbers simultaneously — perfect for matrix math.

VRAM (Video RAM) is the GPU's own memory — separate from CPU RAM. Data must be explicitly transferred from CPU RAM to VRAM before the GPU can operate on it. This transfer happens over the PCIe bus (~16 GB/s bandwidth). If your program constantly transfers large amounts of data per batch, that transfer time dominates — the GPU sits idle waiting for data.

The RTX 5080 Laptop GPU has 16 GB of VRAM. Our training matrix (sparse) is only about 120 MB on the GPU — tiny. The bottleneck was never the computation itself.

5.3 The Original Code — Where the Bottleneck Was

The original knn_predict loop looked like this:

```
for start in range(0, n_test, batch_size):        # loop over batches
    chunk = test_mat[start:end].toarray()          # [CPU] sparse → dense:
zero-fill 1.2 GB
    batch = torch.tensor(chunk, device=DEVICE)     # [CPU→GPU] transfer
1.2 GB over PCIe
    sims  = torch.sparse.mm(train_gpu, batch.T).T  # [GPU] matrix multiply
— fast
    ...                                            # [GPU→CPU] transfer
tiny top-k results
```

The timeline per batch looked like this:

```
|▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓|  CPU (toarray + PCIe
transfer)
|                                        ▓▓▓▓|  GPU (matmul)
```

```
GPU active = ~20% of total time. 80% is the CPU+PCIe bottleneck.
```

.toarray() on a 2000×150,000 sparse matrix must allocate and zero-fill a 2000×150,000 float32 array = 1.2 GB, even though a news article only touches ~200–500 of those 150,000 columns. Then it ships all 1.2 GB over PCIe. By the time the transfer finishes, the GPU has been idle for the entire duration.

5.4 How We Diagnosed It

We observed the GPU utilization percentage in Task Manager while the code was running. It showed ~20% utilization even with batch_size=2000. The pattern was clear: utilization spiked briefly then dropped to near-zero repeatedly, in sync with the batch loop. That "sawtooth" pattern is the signature of a CPU-bound transfer bottleneck — the GPU is starved for data.

We also noticed that when running a denser model (char n-grams, ~7x more non-zeros per document), the CPU utilization dropped and GPU utilization rose — confirming the bottleneck was the CPU zero-filling work, not the GPU matmul.

5.5 First Attempt: Pre-Loading Test Data as float16

Our first fix attempt: load the ENTIRE test matrix onto GPU before the loop, storing in float16 to halve VRAM usage.

```
# Attempt 1: pre-load entire test set as float16
PREFETCH = 5000
parts = []
for i in range(0, n_test, PREFETCH):
    chunk = test_mat[i:i+PREFETCH].toarray()                          #
CPU sparse → dense
    parts.append(torch.tensor(chunk, dtype=torch.float16, device=DEVICE))
# → GPU float16
test_gpu = torch.cat(parts, dim=0)  # full test matrix lives on GPU

for start in range(0, n_test, batch_size):
    batch = test_gpu[start:end].to(torch.float32)   # GPU slice + upcast
(nanoseconds)
    sims  = torch.sparse.mm(train_gpu, batch.T).T   # GPU matmul
```

Theory: A 25,520×150,000 test matrix in float32 = 15.3 GB (exceeds 16 GB VRAM). In float16 it is 7.7 GB — fits. The float16→float32 upcast per batch is a pure GPU operation taking microseconds. One-time transfer instead of per-batch transfer.

Reality: On Windows, .toarray() on chunks of 5,000 rows required ~3 GB CPU RAM allocations per chunk. The chunked CPU allocation was actually slower than the original per-batch approach — we were waiting longer, not shorter. Also, the code was also accidentally re-uploading the train matrix inside every knn_predict call (26 times during a k sweep), compounding the slowdown. Reverted.

5.6 Root Cause #2: Train Matrix Re-Uploaded 26x Per Model

While investigating the slowdown, we found a second bottleneck: the training matrix was being converted and uploaded to the GPU inside every knn_predict call. During a k sweep with 22 k-values, this meant 22 separate train uploads per model. The fix: upload once in best_k_search and pass it as a parameter.

```
def best_k_search(val_mat, fit_mat, fit_labels, val_labels, k_list):
    train_gpu = scipy_sparse_to_torch(fit_mat, DEVICE)  # upload ONCE
    for k in k_list:
        preds = knn_predict(..., _train_gpu=train_gpu)   # reuse across
all k values
    del train_gpu
    return best_k, best_f1
```

5.7 The Real Fix: Sparse Non-Zero Transfer

The actual solution to the toarray() bottleneck: transfer only the non-zero values and reconstruct the dense matrix on the GPU itself, using its 1.7 TB/s memory bandwidth to fill the zeros — instead of the CPU doing it at ~50 GB/s and shipping 1.2 GB over PCIe.

```
def sparse_batch_to_gpu_dense(batch_csr, device):
    n_rows, n_cols = batch_csr.shape
    data    = torch.tensor(batch_csr.data,    dtype=torch.float32,
device=device)  # non-zeros only
    indices = torch.tensor(batch_csr.indices, dtype=torch.int64,
device=device)  # column indices
    counts  = torch.tensor(np.diff(batch_csr.indptr), dtype=torch.int64,
device=device)
    row_ids = torch.repeat_interleave(
        torch.arange(n_rows, dtype=torch.int64, device=device), counts)
    dense = torch.zeros(n_rows, n_cols, dtype=torch.float32,
device=device)  # GPU zero-fill
    dense[row_ids, indices] = data  # scatter non-zeros into place
    return dense
```

The performance difference per batch:

```
Old way:  zero-fill 1.2 GB on CPU at ~50 GB/s   ≈ 24 ms
          + transfer 1.2 GB over PCIe            ≈ 19 ms  → ~43 ms total

New way:  transfer ~1.2 MB of non-zeros          ≈ 0.02 ms
          + torch.zeros on GPU at ~1.7 TB/s      ≈ 0.7 ms  → ~0.7 ms total

~40x less time per batch. 1000x less PCIe traffic. Same numerical results.
```

With this fix, lighter models (M1–M7, ~200–300 non-zeros per doc) became CPU-bound at ~90% CPU utilization — meaning the CPU was now the bottleneck, which is expected and acceptable. The denser char n-gram model was GPU-bound. Both are the correct behavior for their respective densities.

5.8 CuPy SpGEMM — Attempted, Blocked by CUDA Toolkit Version

The next logical step was CuPy SpGEMM: sparse×sparse GPU matrix multiplication via cuSPARSE. This would eliminate the dense matrix creation entirely — both train and test

batch stay sparse on the GPU, and only the result (similarity scores) is materialized as dense for the topk operation.

```
# CuPy SpGEMM approach
train_gpu = cupy_csr(...)          # sparse on GPU
batch_gpu = cupy_csr(...)          # sparse on GPU — only non-zeros
transferred
sims_cp   = (train_gpu @ batch_gpu.T).toarray()  # sparse×sparse → dense
result only
sims      = torch.as_tensor(sims_cp, device=DEVICE).T  # zero-copy
CuPy→PyTorch
```

CuPy confirmed working on RTX 5080 + CUDA 13 driver. However, when the transpose (.T) operation on a CuPy sparse matrix triggered an NVRTC kernel compilation to check canonical format, it failed with a compilation error. Root cause: CuPy's bundled libcudacxx headers reference __nv_fp8_e8m0 (an FP8 type added in CUDA 12.8+), but the installed CUDA toolkit on disk was v12.6 — incompatible. The CUDA driver version (13) sets a ceiling; it does not mean the toolkit is 13.

A workaround exists (setting _has_canonical_format = True on the CuPy CSR to skip the NVRTC check), but given that the PyTorch sparse×dense approach with sparse_batch_to_gpu_dense() already achieves ~1000x PCIe traffic reduction, the incremental gain from SpGEMM was not worth the added complexity. The PyTorch path remains the final implementation.

PART 6 — THE FULL PROGRESSION: HOW WE GOT HERE

Every score listed is macro F1. Local = 80/20 validation split. LB = leaderboard.

Step 1: First Submission — Baseline

Change: 3 models (M1–M3), k=11, simple BM25 params

Scores: Local: ~0.917 | LB: 0.4036

Started with BM25+trigrams (M1), TF-IDF+bigrams (M2), BM25+unigrams (M3). k=11 fixed, basic weighted voting. Leaderboard score was 0.4036 — shockingly low. Cause: test.dat had no labels so the output format was wrong. Fixed and resubmitted.

Step 2: BM25 Parameter Tuning

Change: Grid search over k1 ∈ {1.2, 1.5, 2.0} × b ∈ {0.5, 0.75, 1.0} on 12k sample

Scores: LB: 0.9490

Found k1=1.2, b=0.5 optimal. Also expanded vocabulary, extended k sweep to range(21,42). Jump from 0.4036 to 0.9490 was mostly the format fix + BM25 tuning.

Step 3: k=11 Bug

Change: K_LIST accidentally set to only range(21,42)

Scores: Local: 0.9186 (dropped)

When extending the k range we dropped k=11 which had been the best for M1. Score dropped. Fixed by explicitly prepending 11: K_LIST = [11] + list(range(21,42)).

Step 4: Expand to 7 Models

Change: Added M4 (BM25+bigrams Porter), M5 (lemma+BM25), M6 (raw words+BM25), M7 (lemma+TF-IDF)

Scores: Local: 0.9201 | LB: 0.9570

Each model brings a different vocabulary and/or scoring. M6 (raw words) was particularly valuable — it keeps word forms like "stocks" and "markets" that stemming conflates. Leaderboard reached 0.9570.

Step 5: Char N-Gram M8 Added

Change: Character 3–4-grams with # boundary markers, chi2(80k)

Scores: Local: 0.9212 | LB: 0.9567 (dropped)

Best local score yet, but leaderboard dropped by 0.0003. The char n-gram features were overfit to the validation split. Removed permanently.

Step 6: Revert to 7 Models, Test Without M3

Change: Removed char n-gram M8; tried dropping M3

Scores: Local: 0.9204 (without M3) | LB: 0.9574 → 0.9573

After reverting M8, leaderboard jumped to 0.9574 — best score so far. Then tried removing M3 (weakest model). Local improved slightly but leaderboard dropped to 0.9573. M3 was restored. Confirmed: M3 is necessary.

Step 7: GPU Bottleneck Fixed

Change: sparse_batch_to_gpu_dense + pre-load train once per k sweep

Scores: Same F1, ~40x faster per batch

Diagnosed 20% GPU utilization from per-batch toarray() + PCIe transfer. Fixed by transferring only non-zeros (~1.2 MB vs 1.2 GB per batch) and filling zeros on the GPU. Also moved train upload outside the 22-iteration k loop.

Step 8: Tried Extended k Sweep

Change: Added k=3,5,7,9 to K_LIST

Scores: Local: marginal gain | LB: overfit risk

Some models found k=3 or k=5 locally optimal but the extended sweep was tuning too tightly to the validation split. Reverted to K_LIST = [11] + range(21,42).

Step 9: CuPy SpGEMM Attempt

Change: Sparse×sparse GPU matmul via cuSPARSE

Scores: Blocked by CUDA toolkit 12.6 vs CuPy libcudacxx requiring 12.8+

Reverted to PyTorch sparse×dense approach which already achieves 1000x PCIe reduction.

Step 10: Expand M1 Features

Change: M1 vocab 200k→300k, chi2 150k→200k

Scores: Local: 0.9203 (M1: 0.9186→0.9216) | LB: 0.9574 (unchanged)

M1 individually improved significantly but the ensemble gain was marginal — the extra trigrams overlap with features already covered by other models. Leaderboard stayed at 0.9574, but the stronger M1 is kept for diversity.

Step 11: Add M8 (TF-IDF + Trigrams)

Change: New M8: TF-IDF scoring on same trigram token set as M1, vocab 300k, chi2(200k)

Scores: Local: 0.9211 (M8 individual: 0.9209) | LB: pending

M8 is the TF-IDF counterpart to M1's BM25. Same features, different term saturation behavior. BM25 and TF-IDF disagree on documents where one class term appears many times — and those are exactly the borderline cases the ensemble needs to resolve. Ensemble improved from 0.9203 → 0.9211 — biggest single gain since M5.

Step 12: Tried Squared F1 Weights

Change: w = best_f1**2 instead of best_f1

Scores: Local: 0.9211 (identical)

With all models scoring 0.913–0.922, squaring the weights barely shifts their relative influence. No votes changed. Reverted to linear weights.

Step 13: Added M9 — LSI on TF-IDF

Change: TruncatedSVD(n_components=300) on train_m8_full, dense cosine k-NN

Scores: Local: 0.9219 (M9 solo: 0.8813) | LB: pending

Introduced latent-semantic space as a fundamentally different view of the documents. M9 solo is lower than any sparse model because 300 latent dims cannot match 200k explicit n-grams for this classification task, but it disagrees with M1–M8 in useful ways. New dense-matrix helpers (knn_predict_dense, best_k_search_dense) added. K_LIST_DENSE = [11,21,31,41,51,71,101] — LSI favors larger k because dense similarity is smoother.

Step 14: Added M10 — LSI on BM25

Change: Second TruncatedSVD(300) on train_m1_full (BM25 + trigrams)

Scores: Local: 0.9220 (M10 solo: 0.8860) | LB: pending

Tried a second LSI view for more diversity. Gain was marginal (+0.0001) because the two SVD-300 models are correlated — they both compress similar underlying n-gram structure. Kept M10 for the small lift, but noted that further LSI variants would be redundant.

Step 15: Score-Level Ensembling

Change: Replaced hard-vote with L1-normalized per-class probability sums

Scores: Local: 0.9240 | LB: pending

Biggest gain of the second phase (+0.0020). Hard voting collapsed each model's confidence into a single label; soft voting preserves the full probability distribution. Added knn_predict_scores and knn_predict_dense_scores that return per-class similarity sums instead of hard labels. Ensemble now weight-sums soft probabilities across all 10 models.

Step 16: Temperature Sharpening Sweep

Change: Swept T ∈ {1.0, 1.5, 2.0, 3.0, 5.0, 7.0, 10.0}

Scores: Local: 0.9240 (best T=1 or 2 — no change)

Tested whether sharpening per-model probabilities before summing would help. Null result: T=1 and T=2 tied at 0.9240. Higher T hurt (T=10 dropped to 0.9223). Interpretation: F1 weights are already doing the right job; sharpening just trades diversity for confidence without net benefit at this model mix.

Step 17: Added M11 — PRF (Aggressive Tuning)

Change: alpha=0.7, beta=0.3, N=20 on M1 (BM25 + trigrams)

Scores: Local: 0.9236 (M11 solo: 0.9182) | LB: pending

First PRF attempt hurt the ensemble. Expanding queries toward the weighted centroid of their 20 nearest neighbors pulled borderline queries into wrong-class territory. M11 solo fell well below M1 (0.9216). Retuned rather than removed — the mechanism is orthogonal to LSI and could still help with the right drift magnitude.

Step 18: PRF — Conservative Re-Tuning

Change: alpha=0.9, beta=0.1, N=10

Scores: Local: 0.9245 (M11 solo: 0.9202) | LB: pending

Conservative tuning: original query dominates (90%), only 10 closest neighbors contribute a small 10% nudge. M11 solo rose to 0.9202 (still just below M1) but the ensemble climbed to 0.9245 (+0.0005 over Step 15). T=1.5 became the optimal temperature with 11 models — a small sharpening now pays off because the weaker LSI models (0.88) dilute more when there are more models overall. This is our current best local score.

Leaderboard Milestones

- 0.4036 — First submission — output format error (later fixed).
- 0.9490 — After BM25 parameter tuning, extended k sweep, larger vocab.
- 0.9570 — 7-model ensemble (M1–M7).
- 0.9567 — After adding char n-gram M8 — went DOWN. Reverted.
- 0.9574 — Best score — 7-model ensemble with M3 restored, M1 at 200k features.
- 0.9573 — After removing M3 — went DOWN. M3 restored.
- pending — 8-model ensemble with TF-IDF+trigram M8. Local: 0.9211.
- pending — 11-model ensemble (+ M9 LSI-TFIDF, M10 LSI-BM25, M11 PRF) with score-level soft voting and T=1.5 temperature. Local: 0.9245.

PART 7 — CODE STRUCTURE OVERVIEW

Cell 1 — Imports: Loads libraries: numpy, scipy, torch, sklearn, nltk. Detects GPU.

Cell 2 — Load Data: Reads train.dat (label + text per line) and test.dat (text only). 102,080 training samples, 25,520 test samples, 4 classes.

Cell 3 — Preprocess (Porter + trigrams): Defines preprocess() using Porter stemmer + unigrams + bigrams + trigrams. Used by M1, M2, M3, M4, M8.

Cell 4 — Apply Preprocess: Converts all train and test texts to token lists. Run once — slow step.

Cell 5 — Helper Functions: Core reusable functions: renormalize, build_vocab_idf, build_bm25_matrix, build_tfidf_matrix, apply_chi2, scipy_sparse_to_torch, sparse_batch_to_gpu_dense (the key GPU optimization), knn_predict (GPU-accelerated with distance-weighted voting), best_k_search.

Cell 6 — BM25 Param Search: Grid search over k1 ∈ {1.2,1.5,2.0} × b ∈ {0.5,0.75,1.0} on 12k-sample subset. Sets BM25_K1=1.2 and BM25_B=0.5.

Cell 7 — Build M1: BM25 + trigrams + chi2(200k), vocab 300k. Shape: (102080, 200000).

Cell 8 — Build M2: TF-IDF + bigrams + chi2(120k). Shape: (102080, 120000).

Cell 9 — Build M3: BM25 + unigrams only + chi2(50k). Shape: (102080, ~35566).

Cell 9b — Build M4: BM25 + bigrams (Porter) + chi2(120k). Shape: (102080, 120000).

Cell 9c — Build M5: Lemma + BM25 + bigrams + chi2(120k). Defines preprocess_lemma() using WordNet.

Cell 9d — Build M6: Raw words + BM25 + bigrams + chi2(120k). Defines preprocess_raw() — no stemming.

Cell 9e — Build M7: Lemma + TF-IDF + bigrams + chi2(120k). Reuses M5 token lists.

Cell 9f — Build M8: TF-IDF + trigrams + chi2(200k), vocab 300k. Reuses train_tokens from Cell 4. Shape: (102080, 200000). The TF-IDF counterpart to M1.

Cell 9g — Build M9 (LSI TF-IDF): TruncatedSVD(n_components=300, n_iter=5) on train_m8_full. Produces dense (102080, 300) matrix. L2-normalized for cosine via dot product. Defines knn_predict_dense and best_k_search_dense for dense GPU k-NN using batch @ train.T.

Cell 9h — Build M10 (LSI BM25): Same as 9g but operating on train_m1_full (BM25 + trigrams) with a different random seed. Shape: (102080, 300).

Cell 9i — Score-Level Helpers: Defines knn_predict_scores and knn_predict_dense_scores — variants that return per-class similarity sums (shape n × 4) instead of hard labels. Also defines l1_normalize_rows for turning sums into probabilities. Enables soft ensembling.

Cell 9j — Pseudo-Relevance Feedback: Defines build_prf_queries. For each query, runs 1st-pass k-NN, takes top-N neighbors weighted by similarity, computes sparse centroid via W_batch @ corpus_mat, then builds expanded_query = alpha * original + beta * centroid. Final tuning: alpha=0.9, beta=0.1, N=10.

Cell 10 — Validate All + Ensemble: Sweeps k for each of 11 models on 80/20 validation split (K_LIST_DENSE for M9/M10, K_LIST for rest). Builds M11 queries via PRF on the validation slice. Collects per-class score matrices from all 11 models, L1-normalizes, and sweeps temperature T. Reports per-model F1 and best ensemble F1.

Cell 11 — Final Predictions: Runs 11-model score-level ensemble on full test set using best k values and best T from Cell 10. Builds test_m1_prf via build_prf_queries(test_m1, train_m1) before M11 inference.

Cell 12 — Save Output: Writes predictions.dat (one label per line). Verifies line count matches format.dat.

PART 8 — KEY TAKEAWAYS

- k-NN accuracy depends almost entirely on the quality of the feature representation — the algorithm itself is simple, the features do all the work.
- BM25 outperforms raw TF-IDF individually, but TF-IDF adds ensemble diversity because the two functions saturate high-frequency terms differently and disagree on borderline documents.
- Chi-squared feature selection removes noise and reduces memory — both improve accuracy. k-NN is especially sensitive to irrelevant dimensions because they dilute cosine similarity.
- Ensembles work because each model makes different errors. Even a weak model (M3, F1=0.913) can help the ensemble if it captures something the other models miss — we confirmed this empirically by observing a leaderboard drop when M3 was removed.
- Local validation F1 is not always a reliable proxy for leaderboard score. We had multiple cases where local improvements hurt the leaderboard (char n-grams, extended k sweep) and a case where a local regression helped (keeping M3). With only 5 submissions per day, being conservative and testing carefully matters.
- GPU utilization is not automatic — the bottleneck is almost always data transfer, not computation. Transferring only non-zero values instead of dense arrays gave ~40x speedup per batch with 1000x less PCIe traffic.
- Diagnosing bottlenecks requires observation, not guessing. Watching GPU utilization in Task Manager revealed the sawtooth idle pattern immediately. The fix followed directly from understanding the cause.
- CUDA driver version ≠ CUDA toolkit version. The driver sets a ceiling for what is supported; the toolkit actually installed on disk determines what compilers and headers are available. A mismatch can silently cause library incompatibilities.
- Trial and error is a valid methodology — but keep good records of what you tried and what happened. Several of our improvements came from reverting failed experiments, which is only possible if you tracked what changed.
- A weak individual model can still help an ensemble. M9 (0.8813) and M10 (0.8860) both score lower than every sparse model, yet including them lifts the ensemble because

their mistakes are different — they disagree with M1–M8 in useful places. Ensemble strength comes from decorrelated errors, not just individually strong models.

- Hard-vote ensembles discard per-query confidence; score-level ensembling preserves it. Switching from "each model votes for one class weighted by F1" to "each model contributes a probability distribution weighted by F1" gave us the single biggest gain of the second phase (+0.002). Whenever possible, combine at the score level, not the label level.
- Temperature sharpening is not always helpful. For well-weighted ensembles with clustered F1 scores, raising probabilities to a power just trades diversity for confidence without net benefit. The F1-based weights were already doing the right calibration.
- PRF requires conservative tuning on clean corpora. Aggressive query expansion (alpha=0.7, beta=0.3) worked well in classic IR papers on noisy query logs, but modern cleanly-written text classification needs timid drift (alpha=0.9, beta=0.1) to avoid pulling queries into wrong-class clusters. Hyperparameter defaults from one domain rarely transfer unchanged to another.
- LSI captures different signal than sparse n-grams, but is not a free win. A rank-300 SVD retains only ~10% of the variance of a TF-IDF matrix — 300 latent dims cannot match 200k explicit features for nearest-neighbor retrieval on this corpus. LSI's value is ensemble diversity, not standalone accuracy.
- Dimensionality reduction changes the optimal k. Sparse models peak at k around 30–40; dense LSI models peak at k around 50–100 because dense cosine similarity is smoother than sparse (fewer exactly-tied distances). Always sweep k per representation, not per model.